

100% Exhaustive Mapping of Chapter 1 (41 Scripts) & Chapter 2 (Chunking)

1. Executive Overview & Scope

This document provides a **complete 100% exhaustive inventory of all 41 Python scripts** in Chapter 1 (01-DataLoading/) and Chapter 2 (02-DocChunking/) of the official GitHub repository (PacktPublishing/RAG-from-First-Principles), matched directly with the textbook content. It distinguishes exact code matches, repository-only benchmark/utility scripts, and textbook-only code snippets.

2. Chapter 1 Master Inventory: 01-DataLoading (All 41 Scripts)

Subfolder: 01-SimpleTextLoading/ (9 Scripts)

Filename	Purpose & Technical Function	Status	Textbook
01-LoadTxtFileWithLangChain.py	Reads single .txt file using LangChain TextLoader	Exact Match	p. 4-5
02-BuildLangChainDocumentObject.py	Constructs Document objects manually with metadata	Exact Match	p. 5
03-02-SpecifyParams...LangChain.py	DirectoryLoader with glob wildcard and multithreading	Exact Match	p. 7-8
03-03-ChangeTool...LangChain.py	Switches DirectoryLoader loader_cls to TextLoader	Exact Match	p. 8-9
03-04-SkipErrors...LangChain.py	DirectoryLoader with silent_errors=True to bypass unsupported files	Exact Match	p. 9
05-LoadDirectoryDocumentsWithLlamaIndex.py	Batch directory loading via LlamaIndex SimpleDirectoryReader	Exact Match	p. 10-11
06-LlamaIndex-BuildDocumentObject.py	Constructs LlamaIndex Document objects manually with metadata	Exact Match	p. 11-12
07-UsingUnstructured_v1.py	Partitions text with Unstructured; uses vars() for metadata inspection	Exact Match	p. 13-15
07-UsingUnstructured_v2.py	Partitions text with Unstructured; uses __dict__ for element metadata	Exact Match	p. 15

Subfolder: 02-StructuredDocumentLoading/ (6 Scripts)

Filename	Purpose & Technical Function	Status	Textbook
01-LangChain-TextLoader-JSON.py	Loads JSON file as plain string into page_content via TextLoader	Exact Match	p. 15
02-LangChain-JSONLoader-JSON.py	Uses JSONLoader with jq schema query syntax (.mainCharacter)	Exact Match	p. 15-18
03-LangChain-WebBaseLoader.py	Web scraping using WebBaseLoader with BeautifulSoup SoupStrainer	Exact Match	p. 30-33
04-LangChain-UnstructuredMarkdownLoader.py	Parses Markdown files into single or multi-element Document objects	Exact Match	p. 33-35
05-01-Unstrutured-SimpleExample.py	Parses Wikipedia web page via UnstructuredLoader and category metadata	Exact Match	p. 33-34
05-02-Unstrutured-OrganizeParentChildElements.py	Maintains parent-child element hierarchy (Title/Table parent_id linkage)	Exact Match	p. 34-35

Subfolder: 03-ParsingImageAndTextData/ (3 Scripts)

Filename	Purpose & Technical Function	Status	Textbook
01-Unstructured-ReadImages.py	Extracts English text from JPG/PNG images via Unstructured OCR	Exact Match	p. 19-20
02-Unstructured-ReadPPT.py	Partitions PowerPoint PPTX slides into structured text elements	Exact Match	p. 20-22
03-LLM-ReadImagesAndText.py	Converts PPT/PDF to images via pdf2image; parses via GPT-4o Vision multimodal OCR	Exact Match	p. 22-25

Subfolder: 04-PDFFileLoading/ (8 Scripts)

Filename	Purpose & Technical Function	Status	Textbook
01-UsingPyPDF.py	Basic PDF text extraction using LangChain PyPDFLoader	Exact Match	p. 4-6
02-UsingPyMuPDF.py	Fast page, image, link, and metadata extraction via pymupdf (fitz)	Exact Match	p. 6
03-UsingPytesseract+pdf2image.py	Scanned PDF fallback: pdf2image rendering + local pytesseract OCR	Repo Utility	N/A (Repo)
04-UsingLlamaParser.py	Cloud PDF parsing using LlamaParse markdown output & MarkdownElementNodeParser	Exact Match	p. 60-63
05-LangChain-Unstructured-PDF-SimpleDisplay.py	High-res PDF parsing strategy='hi_res' via Unstructured lazy_load()	Exact Match	p. 50-55
06-Unstructured-ParsePDFWithPartitionFunction-v1.py	Partitions PDF file into raw elements using partition()	Exact Match	p. 49-50
06-Unstructured-ParsePDFWithPartitionFunction-v2.py	Partitions PDF and aggregates counts by element type (Title, Text, etc.)	Exact Match	p. 50
08-RenderPDFPageLayout.py	Renders PDF page layout and draws bounding boxes around parsed elements	Exact Match	p. 55-60

Subfolder: 05-TableDataLoading/ (10 Scripts)

Filename	Purpose & Technical Function	Status	Textbook
01-01-ImportCSV.py	Compares row-by-row CSVLoader (custom fieldnames/source_column) vs UnstructuredCSVLoader	Exact Match	p. 26-30
01-02-SpecifyCSVLoaderWhenImportingDirectory.py	DirectoryLoader targeting *.csv files using CSVLoader loader_cls	Exact Match	p. 30
02-01-LlamaIndex-SQLDB.py	Connects LlamaIndex DatabaseReader to MySQL to convert SQL rows to Documents	Exact Match	p. 12-13
02-02-SQLDB-connection-test-pymysql.py	Low-level MySQL connection and query test using PyMySQL	Repo Utility	N/A (Repo)
02-03-SQLDB-connection-test-sqlalchemy.py	Low-level database connection test using SQLAlchemy engine and pandas	Repo Utility	N/A (Repo)
03-01-camelot-ExtractPDFTable.py	Extracts structured tables from PDF files using Camelot-py and Ghostscript	Exact Match	p. 60
04-01-pdfplumber-ExtractPDFTable.py	Extracts tables from PDF pages using pdfplumber into pandas DataFrames	Exact Match	p. 61-62
04-02-pdfplumber-ExtractPDFTableAndQA.py	Extracts PDF tables with pdfplumber, converts to text, and indexes in LlamaIndex	Exact Match	p. 62-63
05-03-unstructured-TableExtractionInferTableStructure.py	Extracts complex PDF tables with infer_table_structure=True	Exact Match	p. 63
06-01-llamaparser-ExtractPDFTable.py	Extracts PDF tables into structured Markdown using LlamaParse cloud service	Exact Match	p. 63-65

Subfolder: 99-Others/ (5 Scripts)

Filename	Purpose & Technical Function	Status	Textbook
99-UsingTextextract.py	Extracts text from multi-format files using Amazon Textract / textract CLI	Repo Utility	p. 14 (Note)
15_LangChain_PDF_Unstructured.py	End-to-end PDF RAG benchmark: UnstructuredPDFLoader + FAISS + RetrievalQA	Repo Benchmark	N/A (Repo)
15_LlamaIndex_PDF.py	End-to-end PDF RAG benchmark: LlamaIndex PDFReader + VectorStoreIndex	Repo Benchmark	N/A (Repo)
15_LlamaParse_PDF_no_Rerank.py	End-to-end PDF RAG benchmark: LlamaParse + MarkdownElementNodeParser (no rerank)	Repo Benchmark	N/A (Repo)
15_LlamaPaser_PDF_Small2Big.py	Advanced PDF benchmark: LlamaParse with Small-to-Big / Parent-Child retrieval	Repo Benchmark	N/A (Repo)

3. Chapter 2 Master Inventory: 02-DocChunking (4 Scripts)

Filename	Purpose & Technical Function	Status	Textbook
01-LangChain-CharacterTextSplitter.py	Fixed character-based text splitting with chunk_size and chunk_overlap	Exact Match	p. 72-75
02-LangChain-RecursivecharacterTextSplitter.py	Hierarchical text splitting prioritizing ['\n\n', '.', '!', ', ']	Exact Match	p. 75-80
03_LlamaIndex-ChunkSizeAffectsAccuracy.py	Evaluates how chunk size affects retrieval precision & QA accuracy in LlamaIndex	Exact Match	p. 88-92
99-Tool-PDF-Splitting.py	Utility script using pypdf (PdfReader/PdfWriter) to slice sample PDF page ranges	Repo Utility	N/A (Repo)

4. Code Snippets in Textbook but NOT Standalone Repo Scripts

The following essential chunking techniques are explained in Chapter 2 text listings with code stamps, but are not saved as standalone .py files in the repository directory:

A. Python Code Splitter (Language.PYTHON)

Preserves function and class block boundaries when chunking source code (Textbook p. 81):

```
from langchain_text_splitters import Language, RecursiveCharacterTextSplitter

python_splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON,
    chunk_size=100,
    chunk_overlap=0
)
python_docs = python_splitter.create_documents([GAME_CODE])
```

B. LlamaIndex SemanticSplitterNodeParser

Adaptively calculates breakpoint percentile thresholds based on embedding similarity (Textbook p. 85):

```
from llama_index.core.node_parser import SemanticSplitterNodeParser
from llama_index.embeddings.huggingface import HuggingFaceEmbedding

embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-zh")
splitter = SemanticSplitterNodeParser(
    buffer_size=1,
    breakpoint_percentile_threshold=95,
    embed_model=embed_model
)
semantic_nodes = splitter.get_nodes_from_documents(documents)
```

C. LlamaIndex SentenceWindowNodeParser

Extracts single-sentence nodes while capturing surrounding window metadata for context expansion (Textbook p. 87):

```
from llama_index.core.node_parser import SentenceWindowNodeParser

node_parser = SentenceWindowNodeParser.from_defaults(
    window_size=3,
    window_metadata_key="window"
)
nodes = node_parser.get_nodes_from_documents(documents)
```

5. Developer Decision Matrix: When to Use Which Code

Data Type / Scenario	Recommended Loader / Parser Code	Key Advantage
Plain Text (.txt, .md)	DirectoryLoader + TextLoader or UnstructuredMarkdownLoader	High speed, simple structure
Structured JSON	JSONLoader with jq_schema (e.g. '.mainCharacter')	Direct field/attribute extraction
Scanned PDFs / Images	pdf2image + pytesseract OCR or GPT-4o Vision	Extracts unsearchable visual text
Complex PDF Layouts	UnstructuredLoader (strategy='hi_res') or LlamaParse	Preserves multi-column layout & tables
PDF Tables	Camelot-py, pdfplumber, or LlamaParse	Converts grid tables to DataFrames/Markdown
Source Code	RecursiveCharacterTextSplitter.from_language()	Keeps function/class definitions unbroken
Semantic Segmentation	SemanticSplitterNodeParser or SentenceWindowNodeParser	Groups sentences by thematic embedding similarity